Отчёт: воспроизведение Exclusive Self Attention

Статья: S. Zhai, «Exclusive Self Attention», arXiv:2603.09078. Код: `xsa_colab.ipynb`, сырые результаты: `results/`.

33 запуска обучения по 100M токенов, бенчмарк блока и анализ готовых моделей. Всё на одной A100 80GB в Colab, около двух часов.

Что воспроизвелось

Утверждение статьи	У нас	Итог
XSA почти не добавляет времени и памяти (рис. 2)	код из статьи: +5–12% времени блока и +4% памяти; через `torch.compile`: +1–4%; шаг обучения +2–4%	воспроизводится
Выход внимания похож на свой value (рис. 1)	cos(y_i, v_i) около 0.4 у GPT-2 и Pythia; роста с глубиной у Pythia-1.4B нет	воспроизводится, кроме роста по слоям
XSA снижает loss (рис. 3)	val loss ниже на 0.23 ± 0.06 на 3 сидах; XSA лучше во всех 18 парах	воспроизводится, эффект на порядок сильнее
Выигрыш растёт с размером модели (рис. 3, табл. 2)	Δ = −0.08, −0.17, −0.54 для моделей на 3M, 11M, 25M параметров без эмбеддингов	направление то же, но у большой модели baseline страдает от высокого LR
Выигрыш не зависит от LR (рис. 4)	Δ = −0.10, −0.17, −0.63 при LR 1e-3, 2e-3, 4e-3	XSA лучше при всех LR, но разница растёт с LR
Выигрыш растёт с длиной контекста (рис. 5)	Δ = −0.23, −0.23, −0.24 при T = 256, 1024, 4096, по 3 сида	не воспроизводится
XSA совместим с attention sinks (рис. 6)	Δ = −0.17, −0.21, −0.27 при 0, 1, 4 sink-токенах	воспроизводится
Выигрыш на downstream (табл. 2)	LAMBADA 0.1% → 0.9%, HellaSwag на уровне случайного	на таком масштабе не проверить

Главное, чего нет в статье: baseline заметно чувствительнее к learning rate, чем XSA. При LR 4e-3 baseline учится гораздо медленнее, а XSA почти так же, как при 2e-3. Отсюда, видимо, и большая часть разницы на нашем масштабе. Но выигрыш остаётся и при лучшем для baseline LR.

Статья

В self-attention выход для позиции i равен $y_i = \Sigma_j a_{ij} v_j$. Автор замечает, что в обученной модели y_i сильно похож на собственный v_i: средний косинус доходит до 0.5 (attention similarity bias). Информация о самом токене и так проходит дальше по residual-связи, поэтому, по мнению автора, attention тратит часть ёмкости впустую.

XSA (exclusive self attention) убирает из y_i компоненту вдоль v_i:

$$z_i = y_i - (y_i^T v_i / \|v_i\|^2) \cdot v_i$$

Параметров не добавляется, в коде это две строки после `scaled_dot_product_attention`. Автор учил модели 0.7B, 1.4B и 2.7B на 100B токенах FineWeb (nanoGPT с RoPE и LayerNorm после эмбеддингов) и сообщает:

1. XSA почти не замедляет обучение и почти не добавляет памяти (рис. 2, B200).
2. Train и val loss ниже на всех трёх размерах (рис. 3), downstream в среднем лучше на 0.3–1.4 пункта, причём с размером модели выигрыш растёт (табл. 2).
3. Разница почти одинакова при разных LR (рис. 4) и растёт с длиной контекста от 512 до 16384 (рис. 5).
4. XSA работает и вместе с attention sinks (рис. 6).

Установка

	Статья	**Здесь**
Модель	0.7B–2.7B без эмбеддингов, 24–32 слоя, head_dim 128–256	основная S: 10.6M без эмбеддингов (30M всего), 6 слоёв, d=384, head_dim 64; ещё XS (4 слоя, d=256) и M (8 слоёв, d=512)
Данные	FineWeb, 100B токенов	FineWeb, 100M токенов на запуск
Шаг	0.5M токенов, 200K шагов	131k токенов, 762 шага
Контекст	2048, в одном эксперименте 512–16384	1024, ещё 256 и 4096
LR	подобран по сетке для baseline	2e-3; для S ещё 1e-3 и 4e-3
Железо	B200	A100 80GB, bf16, `torch.compile`
Повторы	1 запуск	3 сида для основной модели и всех длин контекста, 1 сид для LR, sinks и размеров

Модель и оптимизатор как в nanoGPT: pre-LN, GELU, связанные эмбеддинги, AdamW (0.9/0.95, weight decay 0.1), warmup 5%, косинус до 0.1·LR. Изменения из статьи (RoPE, LayerNorm после эмбеддингов) тоже есть. Sink-токены сделаны как у Xiao et al.: обучаемые векторы в начале последовательности, на их позициях loss не считается.

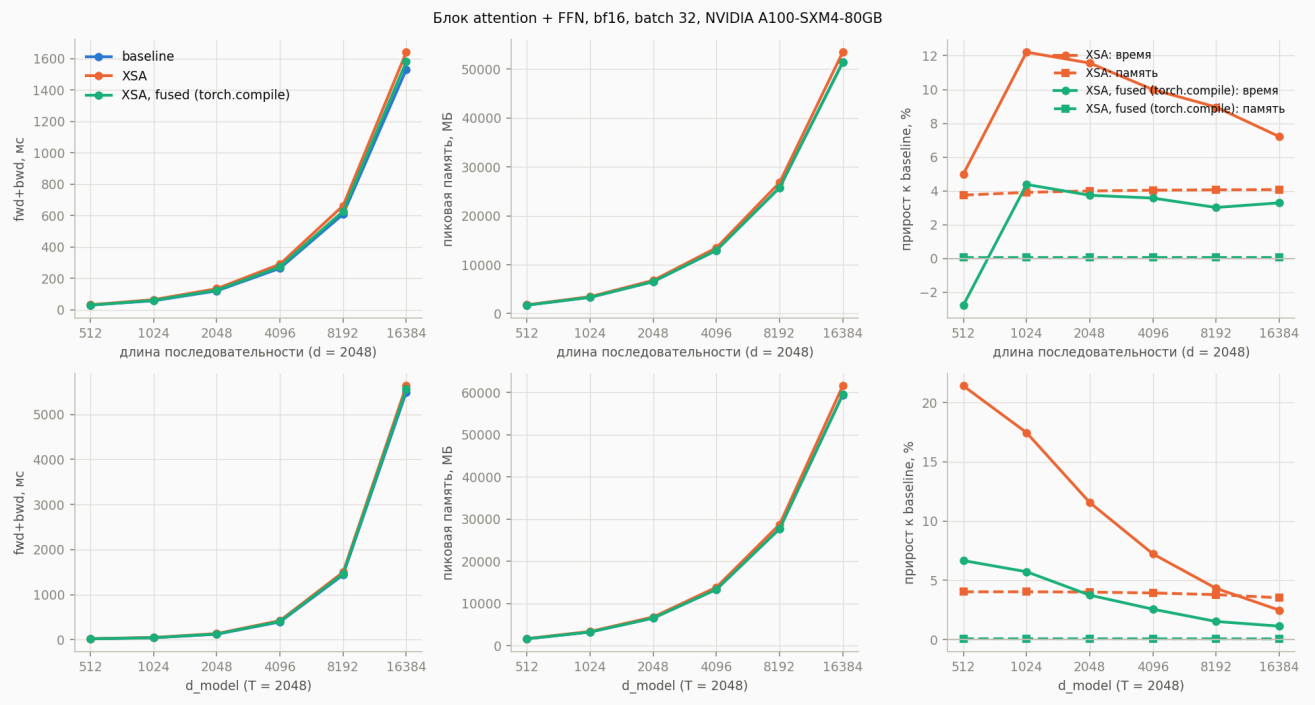
Сравнение парное. При одном сиде baseline и XSA стартуют с одинаковых весов (XSA параметров не добавляет) и видят одни и те же батчи в одном порядке. У разных сидов разная инициализация и разные 100M токенов. Итоговый val loss считается на 10M токенов валидационного шарда.

Реализацию проверил тестом: она совпадает с формулой, посчитанной циклом по позициям, z_i ортогонален v_i, градиент сходится с численным. Попутно выяснилось, что на первой позиции y_1 = v_1, поэтому у XSA выход внимания там ровно ноль.

Результаты

1. Время и память

Блок attention+FFN, forward+backward в bf16, batch 32, head_dim 128, как на рис. 2 статьи. Сравнивал три варианта: baseline, XSA из статьи (eager) и ту же функцию, собранную `torch.compile` в одно ядро (fused).



	d=2048, T от 512 до 16384	T=2048, d от 512 до 16384
XSA eager, время	+5…+12%	+21% → +2.5%
XSA fused, время	−3…+4%	+7% → +1%
XSA eager, память	+4%	+4%
XSA fused, память	+0.1%	+0.1%

XSA добавляет поэлементные операции объёмом O(B·T·d), а матричные умножения в блоке стоят O(B·T·d²). Поэтому относительный оверхед падает с ростом d. В статье на B200 при T=16384 разница на глаз около 10% по времени и 8% по памяти, то есть того же порядка, что и в eager-варианте здесь.

Сама операция упирается в память. Минимальный трафик для неё 8 проходов по тензору B·T·d: forward читает y, v и пишет z, backward читает y, v, dz и пишет dy, dv. Eager-версия запускает несколько отдельных ядер (normalize, умножение, сумма, умножение, вычитание), каждое читает и пишет полный тензор, и сохраняет нормированный v для backward. При T=16384 и d=2048 она идёт 89 мс, а fused 22 мс. В пересчёте на минимальный трафик это 190 и 780 ГБ/с при пике A100 около 2 ТБ/с. Прирост памяти тоже объясняется этим тензором: при T=2048 и d=2048 eager-версия берёт на 262 МиБ больше, а один тензор B×T×d в bf16 занимает 256 МиБ.

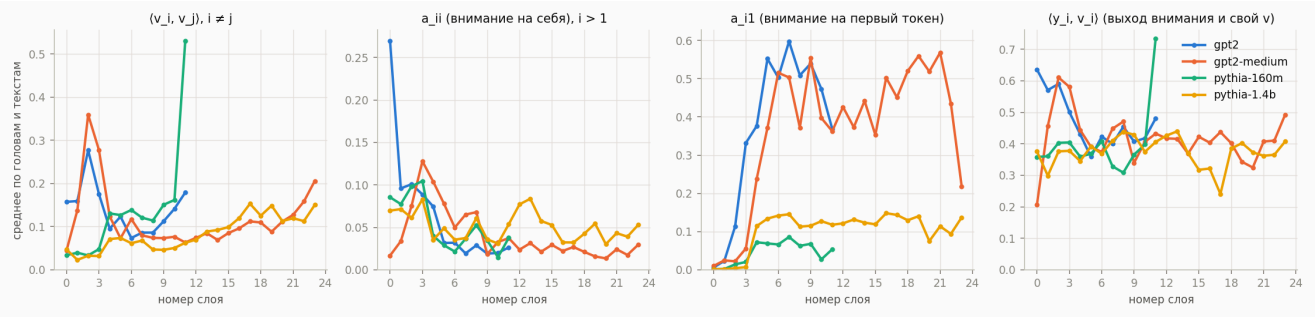
При обучении вся модель скомпилирована, так что XSA там уже fused:

Модель	baseline, мс/шаг	XSA, мс/шаг	время	пиковая память
XS, T=1024	121.0	124.2	+2.7%	+2.7%
S, T=256	185.5	192.3	+3.7%	+4.4%
S, T=1024	195.6	202.4	+3.4%	+4.4%
S, T=4096	238.1	242.8	+2.0%	+4.4%
M, T=1024	305.6	317.4	+3.9%	+5.5%
S, XSA только в слоях 3–5	195.6	199.0	+1.7%	+2.2%

У маленьких моделей d от 256 до 512, поэтому оверхед заметнее, чем на больших d в бенчмарке. С ростом T время шага растёт за счёт внимания, и доля XSA падает.

2. Attention similarity bias

Модель автора не выложена. Ближе всего к ней Pythia-1.4B: 24 слоя, RoPE, контекст 2048. Ещё смотрел Pythia-160M, GPT-2 и GPT-2 medium. По 64 последовательности из валидации FineWeb, усреднение по головам и текстам.

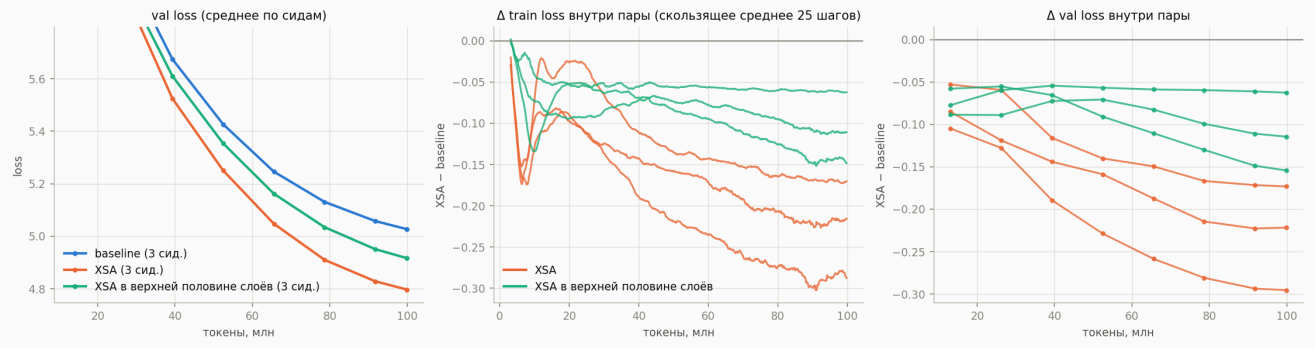


Модель	cos(y_i, v_i), среднее по слоям	по слоям	cos(v_i, v_j)	a_ii
GPT-2	0.47	0.36–0.64	0.14	0.07
GPT-2 medium	0.42	0.21–0.61	0.12	0.04
Pythia-160M	0.40	0.31–0.73	0.14	0.05
Pythia-1.4B	0.38	0.24–0.44	0.08	0.05
1.3B из статьи		0.2–0.55	0.04–0.10	0.02–0.13

Эффект воспроизводится. У всех четырёх моделей выход внимания заметно сонаправлен со своим v: косинус около 0.4. У случайных векторов размерности 64–128 он был бы около 0 ± 0.1. Рост cos(y_i, v_i) с глубиной, который есть у автора, у Pythia-1.4B не виден: по слоям он держится в 0.3–0.45.

3. Loss

Основная модель S, T=1024, LR 2e-3. Кроме XSA во всех слоях, проверил вариант с XSA только в верхней половине слоёв (3–5). Так XSA ставили в OpenAI Parameter Golf, в статье этого варианта нет.



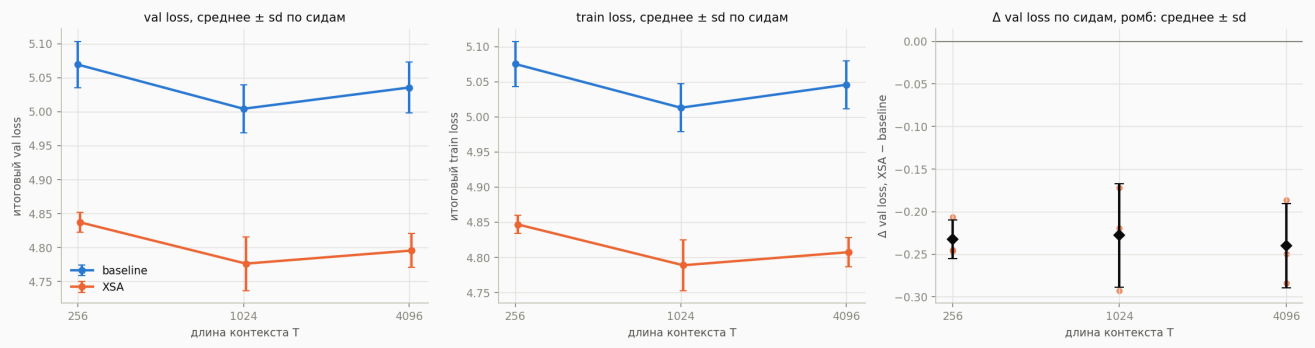
Сид	baseline	XSA	XSA в слоях 3–5	Δ XSA	Δ XSA в слоях 3–5
0	4.994	4.822	4.932	−0.172	−0.062
1	5.044	4.751	4.893	−0.293	−0.151
2	4.975	4.756	4.862	−0.220	−0.114
среднее	5.004	4.776	4.895	−0.228 ± 0.061	−0.109 ± 0.045

Итоговый val loss. XSA лучше на всех трёх сидах, p = 0.02 по t-тесту для Δ (при трёх сидах это грубая оценка). Train loss ведёт себя так же: Δ = −0.224 ± 0.057. XSA в половине слоёв даёт половину выигрыша при половине оверхеда. Всего в полном прогоне 18 пар baseline–XSA с одинаковыми весами и данными, и XSA лучше во всех.

Разница растёт по ходу обучения: на 13M токенов она от −0.05 до −0.10, на 100M от −0.17 до −0.30. Расти она перестаёт только в самом конце, когда LR уже почти затух.

У автора разница в loss около 0.02–0.03, здесь на порядок больше. Отчасти это объясняет раздел 5: baseline сильнее проседает от высокого LR. Остальное, видимо, дают масштаб (модель в 70–250 раз меньше по параметрам без эмбеддингов) и в 1000 раз более короткое обучение.

4. Длина контекста



T	baseline	XSA	Δ
256	5.069	4.837	−0.232 ± 0.023
1024	5.004	4.776	−0.228 ± 0.061
4096	5.036	4.796	−0.240 ± 0.049

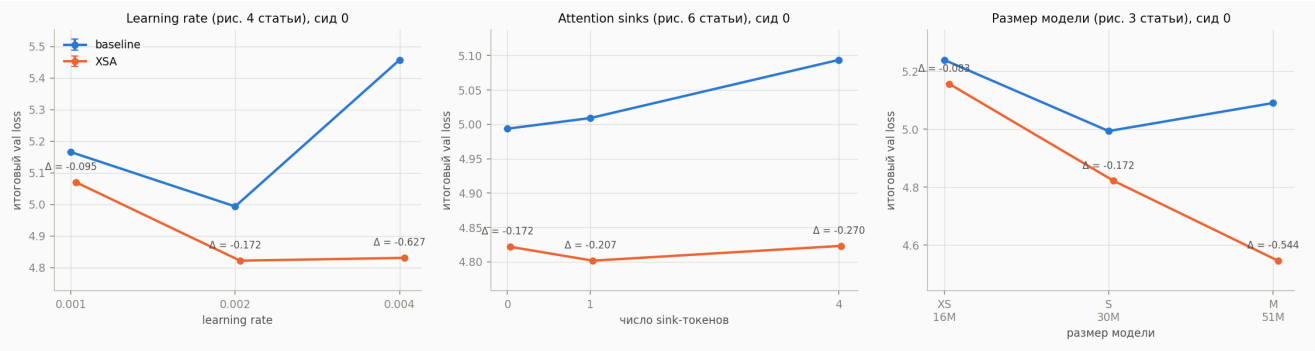
По 3 сида на каждую длину. Выигрыш при всех трёх длинах одинаковый: разница между ними около 0.01, а разброс между сидами 0.02–0.06. Утверждение статьи, что выигрыш растёт с длиной

контекста, здесь не воспроизвелось.

Вероятная причина в длине обучения. На 100M токенов модели ещё плохо используют длинный контекст: loss при T=4096 у обоих вариантов даже выше, чем при 1024. У автора 100B токенов, и там loss с ростом T падает.

5. Learning rate

Модель S, T=1024, сид 0.



LR	baseline	XSA	Δ
1e-3	5.166	5.071	−0.095
2e-3	4.994	4.822	−0.172
4e-3	5.458	4.831	−0.627

XSA лучше при всех трёх LR. Лучший LR для baseline из трёх как раз 2e-3, на нём и учились остальные запуски. Если сравнивать лучшее с лучшим, 4.994 против 4.822, XSA выигрывает 0.17. Так что выигрыш не объясняется неудачным LR для baseline.

Но разница не постоянна, как на рис. 4 статьи. При LR 4e-3 baseline учится заметно медленнее, а XSA почти так же, как при 2e-3. Всплесков loss у baseline нет: кривая просто идёт выше всё обучение. Получается, что XSA делает обучение менее чувствительным к LR, и на нашем масштабе это заметная часть выигрыша.

6. Attention sinks

Модель S, T=1024, сид 0. Перед последовательностью ставится 1 или 4 обучаемых вектора.

sink-токенов	baseline	XSA	Δ
0	4.994	4.822	−0.172
1	5.009	4.802	−0.207
4	5.094	4.823	−0.270

Разрыв сохраняется, как на рис. 6 статьи. Baseline обучаемые sink-токены не помогли: 5.01 и 5.09 против 4.99 без них, при разбросе между сидами около 0.03.

После `quick` у меня была гипотеза: baseline на нашем масштабе не успевает выучить attention sink, а XSA получает его бесплатно (раздел 8), и отсюда большой выигрыш. Этот опыт её не

подтверждает: явный sink baseline не помогает. Оговорка: здесь один сид, и я не проверял, смотрит ли baseline на sink-токены.

7. Размер модели

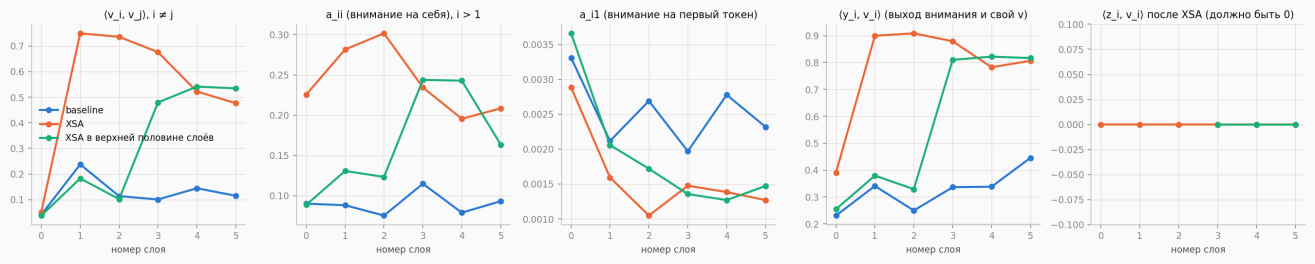
Сид 0, LR 2e-3 для всех размеров.

Модель	параметры без эмбеддингов / всего	baseline	XSA	Δ
XS: 4 слоя, d=256	3.1M / 16M	5.239	5.157	−0.083
S: 6 слоёв, d=384	10.6M / 30M	4.994	4.822	−0.172
M: 8 слоёв, d=512	25.2M / 51M	5.090	4.546	−0.544

Выигрыш растёт с размером модели, как в статье. Но к цифре для M есть оговорка: baseline M хуже, чем baseline S, хотя модель больше. Это та же картина, что у S при LR 4e-3. Для M LR 2e-3 великоват, baseline от этого проседает, а XSA нет. Отдельно LR для M я не подбирал.

8. Что выучивает XSA

Те же величины, что в разделе 2, но для наших моделей S после обучения, среднее по трём сидам.



	baseline	XSA	XSA в слоях 3–5: слои 0–2	XSA в слоях 3–5: слои 3–5
a_ii, внимание на себя	0.08–0.12	0.20–0.30	0.09–0.13	0.16–0.24
cos(v_i, v_j)	0.04–0.24	0.05–0.75	0.04–0.18	0.48–0.54
cos(y_i, v_i) до проекции	0.23–0.45	0.39–0.91	0.25–0.38	0.81–0.82
a_i1, внимание на первый токен	0.002–0.003	0.001–0.003	0.002–0.004	0.001

Слои с XSA смотрят на себя в 2–4 раза чаще, а их value-векторы сильно сонаправлены между собой. Всё, что attention набирает вдоль v_i, XSA потом вычитает. Значит, внимание на себя и на похожие токены работает как способ ничего не выдать. Это то, что автор называет неявным attention sink. В модели с XSA в половине слоёв это происходит только в слоях с XSA, нижние слои выглядят как у baseline.

На первый токен не смотрит ни одна из наших моделей (у GPT-2 в среднем 0.37). Но раздел 6 показывает, что дело не только в отсутствии sink: обучаемые sink-токены baseline не помогают.

9. Downstream

	baseline	XSA	XSA в слоях 3–5	случайный уровень
LAMBADA, точность	0.10–0.12%	0.70–1.07%	0.31–0.35%	0
HellaSwag, acc_norm	23.9–24.2%	24.2–24.8%	24.1–24.4%	25%

Диапазоны по трём сидам. Модель на 30M параметров после 100M токенов почти ничего не решает. На LAMBADA XSA лучше на всех трёх сидах, но речь о долях процента. HellaSwag на уровне случайного у всех вариантов. Табл. 2 статьи на этом масштабе не проверить.

Отличия от статьи и ограничения

- Масштаб: до 25M параметров без эмбеддингов и 100M токенов против 0.7–2.7B и 100B.
- LR подбирал только для S и только по трём значениям, для XS и M не подбирал.
- LR, sinks и размеры по одному сиду.
- head_dim 64, у автора 128–256.
- Рис. 1 снят на Pythia и GPT-2, потому что модель автора не выложена.
- Из downstream только LAMBADA и HellaSwag. HellaSwag считался по средней потере на токен окончания, без lm-eval-harness.